TRADUCIR NO ES COPIAR

Traduce el tipo → C#

Los tipos reales de `legado/db/esquema-dbisam.sql` y de `UCalculoTotales.pas` en Pedidos Legado, con su equivalente en C# y el riesgo de cada conversión. PostgreSQL queda como columna opcional.

Legado	C#	PostgreSQL (opcional)	Riesgo de la conversión
`MONEY` PRECIO, TOTAL, FLETE_BASE	`decimal`	`numeric(12,0)`	**Nunca float ni double.** En los CSV vienen como enteros, sin separador de miles.
`Double` en el .pas: PrecioUnitario, Total…	`decimal`	—	**Dinero en punto flotante.** El legado calcula con Double; en C# todo el cálculo va en `decimal`, incluida la tasa de IVA (`0.19`).
`BOOLEAN` ACTIVO	`bool`	`boolean`	En los CSV viene `TRUE` o `FALSE`: revisa cómo lo lees.
`DATE` FECHA, FECHA_VENCIMIENTO	`DateOnly`	`date`	Formato ISO (`2025-01-06`). `FECHA_VENCIMIENTO` puede venir vacía: usa `DateOnly?`.
`CHAR(1)` TIPO (N, V) · ESTADO (A, C)	un `enum`	`char(1)` con `CHECK`	Letras mágicas sin significado: ponle nombre a cada valor.
`CHAR(3)` CATEGORIA	`string` o `enum`	`char(3)`	La categoría `OFE` queda fuera de la base del descuento: es una regla de negocio, no un detalle.
`INTEGER` FOLIO (sin AUTOINC)	`int`	`integer`	El legado asigna el folio con `SELECT MAX(FOLIO) + 1`, que puede repetirse si dos usuarios guardan a la vez. Decide cómo lo generará el sistema nuevo.
`VARCHAR(n)` RAZON_SOCIAL, DIRECCION	`string`	`varchar(n)`	Largo máximo y codificación: hay tildes (`Almacén`); lee los CSV en UTF-8.

EL MISMO DATO, DE PUNTA A PUNTA

DBISAM

```
"TOTAL" MONEY,
"TIPO" CHAR(1),
```

DELPHI

```
Total: Double;
Tipo: Char;
```

C#

```
public decimal Total { get; init; }
public TipoCliente Tipo { get; init; }
```

⚠ **Regla de oro:** el dinero va en `decimal`. Si ves diferencias como `0,0000001`, hay un `double` escondido.

MÁS ALLÁ DE LOS TIPOS

Correspondencia de conceptos

Traducir línea por línea produce «Delphi escrito con sintaxis de C#». Pídele a Claude C# idiomático: cada pieza tiene su forma en .NET.

Delphi	C# y .NET	Qué ganas
`unit` (`.pas`)	Clase o namespace	Código agrupado por responsabilidad.
Formulario (`.dfm`)	Endpoint o vista	La pantalla deja de mezclar reglas y datos.
Evento `OnClick`	Handler o acción del controlador	Una entrada clara por cada acción.
`record` + `array of`	`record` + `List<T>`	Datos con nombre y listas que crecen sin `SetLength`.
`TDBISAMQuery` con SQL concatenado	Repositorio con parámetros	El SQL deja de armarse pegando texto.
Variable global (`UGlobales`)	Configuración inyectada	Se puede probar sin estado escondido.
Letra mágica (`'V'` , `'N'`)	`enum`	El compilador te avisa si te equivocas.

EL REDONDEO, APARTE

El `Calc3` del legado

`Trunc(x + 0.49)` no es un redondeo al peso estándar: es el defecto **R05**. En la etapa 2 se replica tal cual para llegar a 60 de 60; en la etapa 3 lo corriges en un commit aparte.

El `Round()` de Delphi

Es bancario: el .5 va al par más cercano. Su equivalente es `MidpointRounding.ToEven`. Si tu traducción usa `AwayFromZero`, el test lo delata.

AL LEER EL GOLDEN MASTER EN C#

Punto decimal

Los números usan punto: léelos con `CultureInfo.InvariantCulture`.

Las líneas del pedido

Van en una sola celda: `cantidad x precio x categoría`, separadas por `|`. Ejemplo: `5x8210xBEB|30x5000xCON`.

Control de cifras

60 casos. La suma de `total` es **58.202.135**; al corregir R05 pasa a **58.202.144**.

Extensión opcional: PostgreSQL

Con `\copy` el orden de columnas del `CREATE TABLE` debe ser el del CSV. Si usas identity, ajusta la secuencia con `setval` tras cargar los ID.

FUENTES

Microsoft · el tipo decimal en C#
learn.microsoft.com/dotnet/csharp/language-reference/builtin-types/floating-point-numeric-types

Microsoft · Math.Round y MidpointRounding
learn.microsoft.com/dotnet/api/system.math.round

Embarcadero · System.Round
docwiki.embarcadero.com/VCL/XE/en/System.Round

PostgreSQL 16 · tipos numéricos y COPY (opcional)
postgresql.org/docs/16/datatype-numeric.html · postgresql.org/docs/16/sql-copy.html